



ESTANDAR

 SESIÓN 10 · UNIDAD III · WEB 3.0

ERC-20 UN ESTÁNDAR COMPARTIDO

Un token no vale por su código, sino porque miles de aplicaciones ya saben hablarle.
Eso es un estándar.

Blockchain y Web 3.0 · Ingeniería de Sistemas
Universidad de San Buenaventura Medellín

CÓMO VAMOS A USAR LAS TRES HORAS

Se construye SOBRE OpenZeppelin: el estándar ya está escrito y auditado. Hoy se aprende el patrón de aprobación, se despliega en Sepolia y se decide la tokenómica del proyecto.

A**EL ESTÁNDAR Y SU MECÁNICA**

Qué es un EIP/ERC, la interfaz, approve/transferFrom, decimales, extensiones y tokenómica.

~65 min**B****LABORATORIO 10**

Token con tope y quema, canje por aprobación, despliegue a Sepolia y revocación.

~85 min**C****PROYECTO**

Diseñar la tokenómica, o justificar por qué el proyecto NO lleva token.

~30 min



AL SALIR DE AQUÍ

Implementar un token fungible conforme al estándar y entender la mecánica de aprobaciones que usa todo DeFi.

LAS PREGUNTAS QUE LA SESIÓN RESPONDE

- 01** ¿Por qué la interoperabilidad depende de un estándar y no del mejor código?
- 02** ¿Por qué existe approve + transferFrom, y qué riesgo trae?
- 03** ¿Por qué no hay decimales de verdad, y cómo se representa el valor?
- 04** ¿Qué distingue una tokenómica sostenible de un esquema extractivo?

RA3 · Programar contratos aplicando estándares de la industria (ERC-20, OpenZeppelin).



SIGLAS Y TÉRMINOS DE HOY

ERC / EIP

Ethereum Request for Comments / Improvement Proposal

Un EIP propone; cuando define una interfaz de aplicación, se le llama ERC. ERC-20 es el número 20.

ALLOWANCE

autorización

Cuántos tokens de A puede mover un tercero C. La base de approve + transferFrom.

DECIMALES

decimals

Cuántas cifras minúsculas tiene el token. 18 es lo habitual. No hay coma flotante real.

EIP-2612

Permit

Aprobar con una firma, sin una transacción aparte. Ahorra un paso y gas al usuario.

FUNGIBLE

intercambiable

Cada unidad es idéntica a otra: un punto vale igual que cualquier otro punto. Como el dinero.

TRANSFERFROM

transferir desde

Mueve tokens de otra cuenta, dentro del límite que esa cuenta autorizó.

MINT · BURN

acunar · quemar

Crear tokens nuevos (sube el suministro) y destruirlos (lo baja).

TOKENÓMICA

tokenomics

Las reglas económicas del token: cuánto hay, cómo se emite, cómo se destruye, cómo se reparte.

A



APROXIMADAMENTE 65 MINUTOS

EL ESTÁNDAR Y SU MECÁNICA

ERC-20 son seis funciones y dos eventos. Que sean SIEMPRE los mismos es justo lo que hace que un token sirva.



UN ACUERDO, NO UNA LEY

Un EIP (Ethereum Improvement Proposal) es una propuesta pública. Cuando define cómo deben hablar las aplicaciones entre sí, se le llama ERC. Nadie obliga a seguirlos; se siguen porque convienen.

	QUÉ ES	EJEMPLO
EIP	La propuesta, con su número y su discusión abierta.	EIP-20, EIP-721, EIP-2612
ERC	Un EIP que fija una interfaz de aplicación (el «RC» de Request for Comments).	ERC-20 = el EIP-20
El número	Orden de llegada de la propuesta. No hay jerarquía en el número.	20 fue de los primeros

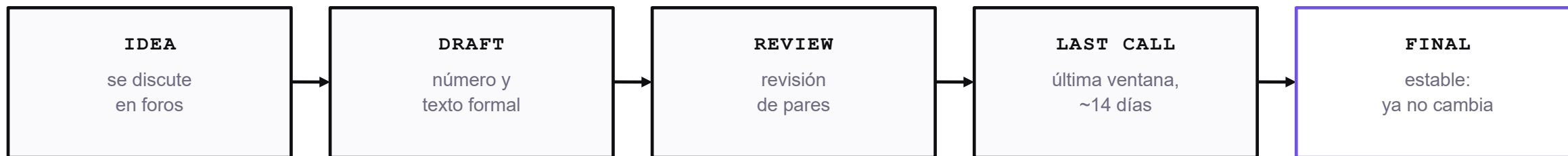
[] POR QUÉ ESTO IMPORTA HOY

ERC-20 (EIP-20) lo propusieron Fabian Vogelsteller y Vitalik Buterin en noviembre de 2015. Está en estado «Final»: es un contrato social estable del que dependen billeteras, exchanges y todo DeFi. Programar contra un estándar Final es programar contra algo que no se va a mover bajo tus pies.



DE UNA IDEA A «FINAL»: EL PROCESO EIP

Un EIP no se aprueba por votación ni lo decreta una empresa. Avanza por estados públicos, definidos en el EIP-1, con discusión abierta y editores que revisan la forma, no el gusto.



Salidas laterales: «Stagnant» (6 meses sin actividad), «Withdrawn» (el autor la retira) y «Living» (documentos que nunca se congelan, como el propio EIP-1).



EJEMPLO REAL Y ERROR TÍPICO

EIP-20 recorrió ese camino y hoy está en «Final». El EIP-2612 (Permit) también. Error típico: citar un EIP en «Draft» como si fuera estándar. Antes de construir sobre un EIP, miren su estado en eips.ethereum.org: si es Draft o Review, la interfaz todavía puede cambiar bajo sus pies.

SEIS FUNCIONES Y DOS EVENTOS

```
interface IERC20 {  
    function totalSupply() external view returns (uint256);  
    function balanceOf(address cuenta) external view returns (uint256);  
    function transfer(address a, uint256 monto) external returns (bool);  
  
    function allowance(address dueno, address gastador) external view returns (uint256);  
    function approve(address gastador, uint256 monto) external returns (bool);  
    function transferFrom(address de, address a, uint256 monto) external returns (bool);  
  
    event Transfer(address indexed de, address indexed a, uint256 monto);  
    event Approval(address indexed dueno, address indexed gastador, uint256 monto);  
}
```

Eso es todo. Un token ERC-20 es cualquier contrato que implemente exactamente estas firmas. OpenZeppelin ya lo hace, bien y auditado: nosotros no reescribimos nada de esto.

QUÉ HACE CADA FUNCIÓN, Y QUIÉN LA LLAMA

FUNCIÓN	QUÉ HACE	QUIÉN LA LLAMA	EL ERROR TÍPICO
<code>totalSupply()</code>	Cuántas unidades existen en total.	Cualquiera (lectura).	Leerla sin dividir por 10^{decimals} .
<code>balanceOf(c)</code>	Cuánto tiene la cuenta c.	Billeteras, exploradores.	Creer que el saldo está en la billetera.
<code>transfer(a, v)</code>	Mueve v de MÍ (msg.sender) hacia a.	El dueño de los tokens.	Mandar a la dirección del propio contrato: se pierden.
<code>approve(g, v)</code>	Autoriza a g a mover hasta v de lo MÍO.	El dueño, antes de usar un contrato.	Aprobar de más «por comodidad».
<code>allowance(d, g)</code>	Cuánto le queda a g por mover de d.	Contratos y apps.	Olvidar que baja al gastarse.
<code>transferFrom(d, a, v)</code>	g mueve v de d hacia a, dentro de lo aprobado.	El gastador (un contrato).	Llamarla sin approve previo: revierte.

Y dos eventos: Transfer (cada movimiento, incluidos acuñar y quemar, con la dirección cero) y Approval (cada autorización). Los exploradores reconstruyen los saldos leyendo esos eventos.



LA TABLA DE SALDOS, ANTES Y DESPUÉS

Todo el token son dos tablas: balances y allowances. Sigamos a Ana, a Beto y al contrato Canje (precio 25) a lo largo de cuatro operaciones. Cifras en FUSB; dentro del contrato, por 10^{18} .

OPERACIÓN	ANA	BETO	TESORERÍA	ALLOWANCE ANA→CANJE	TOTALSUPPLY
0 · <code>acunar(ana, 100)</code>	100	0	0	0	100
1 · <code>ana.transfer(beto, 30)</code>	70	30	0	0	100
2 · <code>ana.approve(canje, 25)</code>	70	30	0	25	100
3 · <code>canje.canjear()</code> (transferFrom)	45	30	25	0	100
4 · <code>beto.burn(10)</code>	45	20	25	0	90

QUÉ HAY QUE NOTAR

`approve` NO mueve tokens (fila 2): solo escribe en la tabla de allowances. Y solo acuñar y quemar cambian `totalSupply`.

UN TOKEN NO ES UNA CUENTA CON MONEDAS DENTRO

El error mental más común: creer que cada usuario «tiene» tokens en su billetera. No es así.

```
mapping(address => uint256) private _balances; // TODO el token es esta tabla
```

DÓNDE VIVE EL TOKEN

En UN contrato: la FichaUSB. Ese contrato guarda una tabla de saldos. Su billetera no contiene tokens: contiene la clave que autoriza a mover una fila de esa tabla.

QUÉ SIGNIFICA «TRANSFERIR»

Restar de una fila y sumar a otra, dentro del mismo contrato. No se mueve nada de un lado a otro: se reescribe una tabla. Por eso una transferencia emite el evento Transfer.

LA CONSECUENCIA PRÁCTICA

Para que una billetera «vea» un token, hay que darle la dirección del contrato: no está en la billetera, está en el contrato.



POR QUÉ 1 TOKEN SON 1 000 000 000 000 000 000

La EVM no tiene coma flotante (Sesión 5). Un token con 18 decimales guarda internamente enteros enormes, y la coma es pura presentación.

LO QUE VE LA PERSONA	LO QUE GUARDA EL CONTRATO (UNIDADES MÍNIMAS)
1 FUSB	1 000 000 000 000 000 000 (1×10^{18})
0,5 FUSB	500 000 000 000 000 000
0,000000000000000001 FUSB	1 (la unidad mínima, indivisible)



EL ERROR DE PRINCIPIANTE

Escribir `acunar(ana, 100)` pensando en 100 tokens, cuando la función espera unidades mínimas, acuña una fracción invisible (100 de 10^{18}). Por eso el token del laboratorio recibe la cantidad EN TOKENS y multiplica por `10**decimals()` adentro: la interfaz habla en tokens, el contrato en unidades mínimas. En `ethers.js` se usa `parseUnits("1", 18)` para escribir y `formatUnits(v, 18)` para leer.



PARSEUNITS Y FORMATUNITS, CON NÚMEROS REALES

En JavaScript nunca se multiplica a mano por 10^{18} : se usan dos funciones de ethers. `parseUnits` va de lo humano al contrato; `formatUnits`, del contrato a lo humano. Salidas reales, ejecutadas con ethers 6:

ETHERS 6 · SALIDA REAL

```
parseUnits("25", 18)      -> 25000000000000000000n    // el precio del Canje
parseUnits("0.1", 18)     -> 10000000000000000000n
parseUnits("1000000", 18) -> 1000000000000000000000000n    // el tope
parseUnits("1.5", 6)      -> 1500000n                // un token de 6 decimales

formatUnits(1234567890000000000000000n, 18) -> "123.456789"
formatUnits(100n, 18)     -> "0.000000000000000001"    // el error de A.4
```



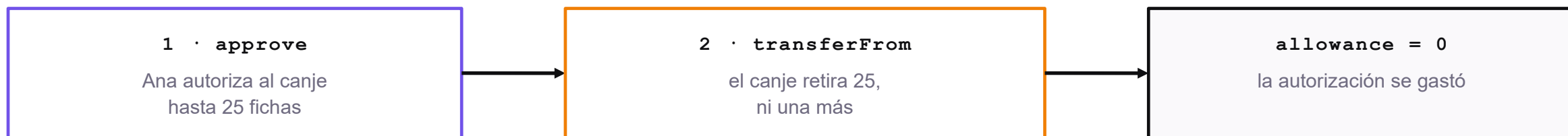
TRES ERRORES TÍPICOS

(1) Usar `Number` en vez de `BigInt`: `25e18` no cabe exacto en un `Number` de JavaScript. (2) Suponer 18 decimales en un token de 6. (3) Pasar un string con coma («0,5»): `parseUnits` exige punto decimal.



POR QUÉ TRANSFERIR EN DOS PASOS

transfer mueve MIS tokens. Pero, ¿cómo deja un usuario que un CONTRATO —un exchange, un canje— mueva los suyos, sin entregarle su clave privada? Con una autorización previa y acotada.



[] LO QUE HACE SEGURO EL PATRÓN

El contrato NUNCA puede mover más de lo aprobado. Verificado en el laboratorio: un segundo canje sin volver a aprobar revierte. La clave privada de Ana no sale nunca de su billetera.

Así funcionan todos los intercambios descentralizados: primero autorizas al contrato del DEX, después el DEX mueve tus tokens al hacer el swap.

QUÉ PASA EXACTAMENTE EN CANJEAR()

```
function canjear() external {
    // el Canje es msg.sender para la ficha: es el GASTADOR
    bool ok = ficha.transferFrom(msg.sender, tesoreria, precio);
    if (!ok) revert TransferenciaFallida();

    premiosDe[msg.sender] += 1;
    emit Canjeado(msg.sender, precio);
}
```

- Ana llama canjear(): para el Canje, msg.sender = Ana.
- El Canje llama a la ficha: para la ficha, msg.sender = el Canje.
- La ficha revisa allowance[Ana][Canje] ≥ 25, la descuenta y mueve 25 de Ana a la tesorería.
- Si no alcanza, revierte TODO: no hay premio sin pago.

EL ERROR TÍPICO DE QUIEN LO PROGRAMA

Escribir `ficha.transferFrom(tesoreria, msg.sender, precio)` —al revés—, o usar `ficha.transfer(tesoreria, precio)`: eso intentaría mover los tokens DEL CANJE, que no tiene ninguno. El «de» de `transferFrom` es el usuario; quien gasta la autorización es el contrato que hace la llamada. Tampoco hace falta un `require` sobre la allowance: `transferFrom` ya revierte con `ERC20InsufficientAllowance`.

APROBACIONES ILIMITADAS: COMODIDAD PELIGROSA

Muchas aplicaciones piden aprobar una cantidad gigante «para no molestar otra vez». Es cómodo, y es el origen de robos enormes.

QUÉ PASA

Si apruebas una cantidad ilimitada a un contrato, y ese contrato resulta malicioso o es hackeado más adelante, puede vaciar TODO tu saldo de ese token cuando quiera. La autorización sigue viva hasta que la revokes.

QUÉ HACER

Aprobar solo lo necesario para la operación. Revisar y revocar aprobaciones viejas (hay herramientas para verlas todas).
Desconfiar de una app que pide aprobación ilimitada sin explicarlo.

EL CASO REAL · BADGER DAO, DICIEMBRE DE 2021

El front-end de Badger DAO fue comprometido (un script malicioso inyectado vía Cloudflare) y pedía a los usuarios firmar aprobaciones ILIMITADAS. Unas 500 billeteras aprobaron; el atacante usó `transferFrom` y drenó del orden de 120 millones de dólares. El contrato estaba bien: el robo entró por una aprobación excesiva que la gente firmó sin mirar. Regla: se entrega la aprobación más pequeña que sirva, y se revoca poniendo la allowance en cero.



¿QUÉ PUEDE HACER EL CANJE CON ESTO?

Ana, «para no aprobar cada vez», autoriza al Canje por `type(uint256).max`. Canjea una vez (25). ¿Cuánto le queda de autorización, y qué podría hacer el Canje si mañana lo reemplazan por uno malicioso?

LA RESPUESTA

Le sigue quedando el máximo: en OpenZeppelin 5, una allowance igual a `type(uint256).max` se trata como INFINITA y no se descuenta al gastarse. El gastador puede mover TODO el saldo de Ana, hoy y dentro de un año.

LA LECCIÓN

Con una aprobación exacta (25), después del canje la allowance queda en 0 y no hay nada que robar. La aprobación ilimitada convierte cualquier fallo futuro del gastador en un robo de tu saldo completo. Es el mecanismo exacto de Badger.



CAMBIAR UNA APROBACIÓN DE 50 A 20

El propio EIP-20 advierte de un riesgo de carrera. Ana aprobó 50 a Beto (un gastador) y decide bajarlo a 20. Parece inofensivo. Pero su transacción pasa por el mempool, a la vista (Sesión 9, A.12):

- | | | |
|-----------|------------------------------------|--|
| 01 | ANA ENVÍA APPROVE(BETO, 20) | Su transacción queda pendiente en el mempool. La allowance todavía vale 50. |
| 02 | BETO LA VE Y SE ADELANTA | Envía <code>transferFrom(ana, beto, 50)</code> con más gas: entra ANTES. Se lleva los 50 viejos. |
| 03 | SE CONFIRMA LA DE ANA | <code>approve</code> escribe 20 (no resta: SOBRESCRIBE). Beto tiene una allowance nueva de 20. |
| 04 | BETO GASTA OTRA VEZ | <code>transferFrom(ana, beto, 20)</code> . Total: 70. Ana quería autorizar como máximo 50. |

[] LA IDEA LLANA

`approve` no dice «cambia de 50 a 20»; dice «ahora son 20», sin saber cuánto se gastó ya.

CÓMO CAMBIAR UNA APROBACIÓN SIN REGALAR

OPCIÓN	CÓMO FUNCIONA	TRADE-OFF
Pasar por 0	approve(g, 0), confirmar, mirar cuánto gastó g, y recién entonces approve(g, M).	Dos transacciones, pero decides M sabiendo lo gastado. Lo recomienda el EIP-20.
increase / decreaseAllowance	Sumar o restar a la allowance en vez de sobrescribirla.	Existían en OpenZeppelin 4. Se QUITARON de ERC20 en OZ 5 (no son del estándar y se usaron en phishing).
SafeERC20 (lado del contrato)	forceApprove y safeIncreaseAllowance, para contratos que aprueban a otros.	Es una librería para quien LLAMA al token, no una función del token.
Aprobar exacto y a tiempo	Aprobar solo lo que se va a gastar, justo antes de usarlo (o con Permit).	La allowance vive poco y vale poco: casi nada que correr.

ERROR TÍPICO: COPIAR UN TUTORIAL VIEJO

Muchos tutoriales llaman token.increaseAllowance(...). Con OpenZeppelin 5.6.1 (el del curso) eso no compila: la función ya no existe en ERC20.



OPENZEPPELIN: COMPONER EXTENSIONES

```
contract FichaUSB is ERC20, ERC20Burnable, ERC20Capped, Ownable {
    constructor(uint256 topeEnTokens)
        ERC20("Ficha USB", "FUSB")
        ERC20Capped(topeEnTokens * 10 ** decimals())
        Ownable(msg.sender)
    {}

    function acunar(address a, uint256 cant) external onlyOwner {
        _mint(a, cant * 10 ** decimals());
    }
    function _update(address from, address to, uint256 v)
        internal override(ERC20, ERC20Capped) { super._update(from, to, v); }
}
```

- ERC20: las seis funciones.
- Burnable: quemar los propios.
- Capped: tope de emisión.
- Ownable: solo el dueño acuña.
- Lo único propio: acunar y el `_update` que compone las extensiones.

El `_update` override es obligatorio en OpenZeppelin v5: al mezclar ERC20 y Capped hay que decir cuál gana. Sin él, no compila — a propósito, para entender que las extensiones se componen aquí.



QUÉ AÑADE CADA PIEZA DE OPENZEPPELIN

EXTENSIÓN	QUÉ AÑADE	CUÁNDO USARLA
ERC20Burnable	burn y burnFrom: destruir tokens propios o aprobados.	Puntos que se canjean o retiran de circulación.
ERC20Capped	Un tope máximo de emisión que nunca se supera.	Cuando prometes escasez: oferta con techo.
ERC20Pausable	Congelar todas las transferencias ante una emergencia.	Si necesitas un «freno» (con su propio riesgo de centralización).
ERC20Permit (EIP-2612)	Aprobar con una FIRMA fuera de la cadena, sin transacción de approve.	Mejor experiencia: una sola transacción hace todo.



COMPONER, NO REESCRIBIR

Se heredan las que se necesitan y se combinan. Cada una está auditada por separado. La habilidad no es escribir un ERC-20 desde cero —eso es un antipatrón— sino elegir y componer las extensiones correctas para tu problema.



BURNABLE, CAPPED Y PAUSABLE: LOS TRADE-OFFS

EXTENSIÓN	LO QUE GANAS	LO QUE CUESTA O ARRIESGA	EJEMPLO EN FICHAUSB
Burnable	Sacar unidades de circulación: canjes, retiros, deflación.	La quema es irreversible; burnFrom exige allowance (otra aprobación que vigilar).	Ana quema 40 → balance y totalSupply bajan 40.
Capped	Escasez creíble: nadie puede emitir por encima del tope, ni el dueño.	Si el tope es muy bajo, el proyecto se queda sin emisión. Cambiarlo exige otro contrato.	Tope 1 000 000: acuñar 1 más revierte con ERC20ExceededCap.
Pausable	Un freno de emergencia: congelar todas las transferencias.	Quien tiene el freno puede congelar el dinero de todos: es poder centralizado.	FichaUSB NO la usa, a propósito: no hay freno que abusar.



ERROR TÍPICO CON CAPPED

Crear que el tope cuenta lo acuñado en la historia. Cuenta el suministro VIVO: si se queman 10 con el tope lleno, se pueden volver a acuñar 10. Lo demuestra la prueba ★ «quemar libera espacio bajo el tope».



EIP-2612 PERMIT: UN PASO MENOS

El patrón clásico exige DOS transacciones: aprobar y luego usar. Permit deja aprobar con una FIRMA fuera de la cadena, que el contrato verifica al usarla. El usuario firma gratis; una sola transacción hace todo.

SIN PERMIT

- Transacción 1: approve (cuesta gas).
- Transacción 2: la operación (cuesta gas).
- Dos confirmaciones en la billetera.

CON PERMIT (EIP-2612)

- Una firma fuera de la cadena (gratis, sin gas).
- Transacción única: la operación incluye la aprobación.
- La firma usa EIP-712, que veremos en la Sesión 13.

OpenZeppelin lo trae en la extensión ERC20Permit. Lo propuso Martin Lundfall en 2020, es estándar «Final», y hoy lo usan casi todos los tokens grandes. En el proyecto es opcional, pero mejora mucho la experiencia.



EL FLUJO DE UNA APROBACIÓN FIRMADA

- | | | |
|-----------|-------------------------------|---|
| 01 | ANA FIRMA, SIN GAS | Su billetera muestra un mensaje estructurado (EIP-712): dueño, gastador, valor, nonce y fecha límite. Firma fuera de la cadena. |
| 02 | LA APP RECIBE LA FIRMA | La firma (v, r, s) viaja a la aplicación. Todavía no ha pasado nada en la cadena. |
| 03 | UNA SOLA TRANSACCIÓN | El contrato llama token.permit(ana, gastador, valor, deadline, v, r, s) y, a continuación, transferFrom. |
| 04 | EL TOKEN VERIFICA | Recupera quién firmó, compara con ana, revisa el deadline y consume el nonce: la misma firma no se puede reusar. |

SI ALGO NO CUADRA

Firma vencida: ERC2612ExpiredSignature. Firmante distinto del dueño: ERC2612InvalidSigner. El DOMAIN_SEPARATOR ata la firma a esa red y a ese contrato.

EL TRADE-OFF

Firmar un permit ES aprobar, aunque no cueste gas. «Es gratis, solo una firma» es justo lo que usa el phishing: se lee el gastador y el valor antes de firmar.



LAS REGLAS ECONÓMICAS SON UNA DECISIÓN DE DISEÑO

DECISIÓN	OPCIONES	QUÉ COMUNICA
Suministro	Fijo (tope) vs. inflacionario (se emite sin fin).	Un tope da escasez creíble; la inflación premia sin fin, pero diluye.
Emisión	Quién acuña y bajo qué regla.	«El dueño imprime cuando quiere» es una señal de alarma.
Quema	Si se pueden destruir tokens, y quién.	La quema reduce el suministro: puede canjear o retirar de circulación.
Distribución inicial	Cómo se reparte el primer lote.	Si el equipo se queda con casi todo, los demás compran su salida.
Vesting	Liberar por partes en el tiempo.	Evita que los primeros vendan todo el día uno.

Un diseño sostenible reparte y limita el poder de emisión. Un esquema extractivo concentra ambos en quien lo lanzó.



EL MISMO CÓDIGO, DOS INTENCIONES

Los dos tokens siguientes son ERC-20 perfectamente válidos, heredados de OpenZeppelin. Lo que los separa no es la calidad del código: es a quién le sirven las reglas.

DECISIÓN	A · SOSTENIBLE (FICHAUSB)	B · EXTRACTIVO (EJEMPLO ILUSTRATIVO)
Suministro	Tope fijo de 1 000 000, visible en cap().	Sin tope: mint() del dueño sin límite.
Emisión	El organizador acuña por asistencia, a la vista (eventos Transfer).	El dueño acuña cuando quiere, sin regla pública.
Distribución	Se reparte a quien participa; el equipo no se queda un lote.	90 % preasignado al equipo, sin vesting.
Quema	Cualquiera quema lo suyo al canjear un premio.	Sin quema; o el dueño quema saldos AJENOS.
Poderes especiales	Ninguno más allá de acuñar bajo el tope.	Pausa, lista negra y comisión de venta que el dueño cambia a su antojo.



LA PREGUNTA QUE DECIDE

¿Quién puede cambiar las reglas después de que tú compraste o recibiste el token?

CÓMO LEER UN TOKEN ANTES DE CONFIAR EN ÉL

Todo esto se puede verificar en el explorador, con el código verificado delante. Verificar en vez de creer (la tesis del curso) aplicado a tokens:

SEÑAL DE ALERTA	CÓMO SE VE EN EL CÓDIGO O EN LA CADENA
El dueño imprime sin tope	Una función de mint con onlyOwner y sin cap(), o con un cap que el dueño cambia.
Concentración inicial	Los primeros eventos Transfer desde 0x0 van casi todos a pocas direcciones.
Sin vesting	El equipo recibe todo el día uno, sin contrato que libere por partes.
Poderes de congelar o confiscar	Pausable, listas negras o funciones que mueven saldos ajenos en manos de una sola llave.
Código no verificado	Si no se puede leer el código en el explorador, no hay nada que auditar: se descarta.

 ERROR TÍPICO

Juzgar el token por el sitio web o por el precio. Se juzga por el contrato y por la distribución real en la cadena.



SEIS IDEAS SOBRE TOKENS

01

Un token vale por cumplir el estándar (EIP-20), no por tener el mejor código.

02

Todo el token es una tabla de saldos dentro de UN contrato.

03

Se hereda y se COMPONE de OpenZeppelin; casi nada se escribe a mano.

04

No hay decimales reales: la interfaz habla en tokens, el contrato en unidades mínimas.

05

approve + transferFrom deja mover tus tokens sin tu clave, y solo lo aprobado; ilimitado es peligroso.

06

La tokenómica es una decisión ética además de técnica: repartir y limitar, o concentrar.



TRES PREGUNTAS ANTES DEL LABORATORIO

PREGUNTA	RESPUESTA ESPERADA
El token tiene 18 decimales. ¿Qué valor debe recibir approve para autorizar 25 fichas?	$25 \times 10^{18} = 25000000000000000000$ (parseUnits("25", 18)).
Ana aprobó 25 al Canje y luego transfirió todo su saldo a Beto. ¿Funciona canjear()?	No: la allowance sigue en 25, pero Ana no tiene saldo. approve no aparta tokens (A.2c).
¿Por qué FichaUSB no hereda Pausable?	Porque el freno sería un poder centralizado sin necesidad en este caso (A.9b).



CÓMO USAR ESTA LÁMINA

Tapar la segunda columna. Una pareja responde cada pregunta en voz alta; otra la corrige. Si la segunda falla en la mayoría del grupo, volver a A.2c antes de abrir el laboratorio: es el error que más tiempo cuesta en el bloque B.

B



APROXIMADAMENTE 85 MINUTOS · EN PAREJAS

LABORATORIO 10

Un token con tope y quema, un contrato que canjea por aprobación, y su despliegue a Sepolia: verificar, importar, transferir y revocar.



FICHAUSB Y CANJE

PARTE	QUÉ HACE	ESTÁNDAR QUE APLICA
FichaUSB	Token de puntos con tope, acuñación controlada y quema.	ERC20 + Capped + Burnable + Ownable
acunar	El organizador crea puntos, hasta el tope.	_mint con control de acceso
Canje	Un puesto que cobra fichas por un premio, usando la aprobación.	approve + transferFrom

```
# Se copia el andamiaje sobre contracts/s10 y se completan los TODO 1-4
npx hardhat test test/s10/FichaUSB.test.js      # 12 pruebas: deben pasar
```



LAS PRUEBAS ★ SON LAS QUE IMPORTAN

No acuñar sobre el tope; quemar libera espacio; el canje retira exactamente lo aprobado; el contrato no puede tomar más. Esas cuatro son la tesis de la sesión.

LOS CUATRO TODO, EN ORDEN

01 TODO 1 · EL TOPE

En el constructor: `ERC20Capped(topeEnTokens * 10 ** decimals())`. Sin convertir, el tope es una fracción invisible.

02 TODO 2 · ACUNAR

`_mint(a, cantidadEnTokens * 10 ** decimals())`. No revisar el tope aquí: lo hace Capped en `_update`.

03 TODO 3 · _UPDATE

Una línea: `super._update(from, to, value)`. Mientras esté vacío, NINGÚN saldo se mueve.

04 TODO 4 · CANJEAR

`transferFrom(msg.sender, tesoreria, precio)`, `revert si false`, sumar el premio y emitir Canjeado (A.5b).

EL ORDEN IMPORTA

Empecen por el TODO 3: sin él casi todas las pruebas fallan, y los otros tres TODO parecen rotos aunque estén bien.



SALIDA ESPERADA, AL EMPEZAR Y AL TERMINAR

```
NPX HARDHAT TEST TEST/S10/FICHAUSB.TEST.JS

# con el andamiaje recién copiado (salida real):
 2 passing
10 failing           # pasan solo nombre/decimales y "solo el organizador acuña"

# con los cuatro TODO completos:
12 passing           # esa es la meta local

# ensayo del despliegue en la red local (salida real):
[ FichaCanjeModulo ] successfully deployed
FichaCanjeModulo#FichaUSB - 0x5FbD...0aa3
FichaCanjeModulo#Canje    - 0xe7f1...0512
```



TERMINADO SIGNIFICA LAS TRES COSAS

(1) 12 pruebas en verde en su máquina. (2) FichaUSB y Canje desplegados en Sepolia con el check verde de verificación en el explorador. (3) Los hashes de transferencia, approve, canjear y el canje fallido tras revocar, anotados.



DEL CONTRATO A UNA DIRECCIÓN REAL

```
# 1. Guardar los secretos en el almacén cifrado (una vez):  
npx hardhat keystore set SEPOLIA_RPC_URL  
npx hardhat keystore set SEPOLIA_PRIVATE_KEY      # billetera DEL CURSO  
npx hardhat keystore set ETHERSCAN_API_KEY  
  
# 2. Desplegar y verificar con Ignition (mismo módulo que la S8):  
npx hardhat ignition deploy ignition/modules/s10-FichaCanje.js \  
  --network sepolia \  
  --parameters ignition/parametros/s10-mi-ficha.json \  
  --verify
```



LAS REGLAS QUE NO CAMBIAN

La clave privada es la de la billetera del curso, SIN fondos reales. Ninguna clave ni URL con API key se escribe en un archivo del repositorio: solo en el almacén cifrado (keystore). Antes de cada git push: git status, y confirmar que no se sube nada. Si la verificación falla por tiempo, esperar un minuto y repetir el MISMO comando: Ignition sabe que ya desplegó.



IMPORTAR, TRANSFERIR Y CANJEAR

01

IMPORTAR EL TOKEN

En MetaMask: «Importar token» con la dirección del contrato. Recordar A.3: no aparece solo.

02

TRANSFERIR

Acuñar puntos a un compañero y que él los vea; guardar el hash de una transferencia entre dos cuentas.

03

APROBAR Y CANJEAR

Desplegar/usar Canje, aprobar el precio desde la billetera, canjear. Revisar que la allowance quedó en cero.

04

REVOCAR

Aprobar de nuevo y luego poner la aprobación en cero: ver que el canje ya NO funciona (revierte).



EL GESTO QUE EVITA LOS ROBOS DE A.6

Antes de firmar cada approve, mirar A QUIÉN se autoriza y CUÁNTO. Aprobar el precio exacto, no una cantidad ilimitada. Revocar cuando ya no se usa es poner la allowance en cero. Es el mismo cuidado que le habría ahorrado 120 millones de dólares a las víctimas de Badger.



SÍNTOMA, CAUSA Y ARREGLO

SÍNTOMA	CAUSA Y ARREGLO
Desplegué y no veo el token en MetaMask.	No falló: hay que importarlo con la dirección del contrato (A.3).
<code>acunar(ana, 100)</code> y aparece 0,0000000000000001.	Falta multiplicar por 10 ** decimals() (TODO 1 y 2).
<code>canjear()</code> revierte con <code>ERC20InsufficientAllowance</code> .	No aprobaron, aprobaron menos del precio, o ya se gastó/revocó. <code>approve</code> del precio exacto.
<code>canjear()</code> revierte con <code>ERC20InsufficientBalance</code> .	Hay allowance pero no saldo: la cuenta no tiene 25 fichas.
<code>acunar</code> revierte con <code>OwnableUnauthorizedAccount</code> .	Se llamó desde una cuenta que no desplegó el token. Solo el organizador acuña.
La verificación en Etherscan falla.	Falta <code>ETHERSCAN_API_KEY</code> en el keystore, o esperar un minuto y repetir el MISMO comando.

La tabla completa, con más casos, está en la guía del laboratorio (sección 10).



EVIDENCIA DEL LABORATORIO 10

EVIDENCIA	DETALLE	PESO
Token verificado	FichaUSB en Sepolia, verificado, importado en la billetera.	35 %
Transferencia	Hash de una transferencia entre dos cuentas del curso.	20 %
Canje completo	Hashes de: approve, canjear, y el intento fallido tras revocar.	30 %
Tokenómica	Un párrafo: ¿por qué eligieron ese tope? ¿Quemarían? ¿Cómo repartirían?	15 %

Todo con hashes verificables en el explorador. Un token que no aparece en la cadena no se califica (política de laboratorios del curso).



C

 APROXIMADAMENTE 30 MINUTOS

EL PROYECTO

Muchos proyectos no necesitan un token. Decidir eso con honestidad vale tanto como diseñar uno bueno.



¿SU PROYECTO LLEVA TOKEN, O NO?

SÍ, SI...

- Hay algo intercambiable que circula entre usuarios.
- Se necesita representar valor, puntos o participación fungibles.
- La interoperabilidad con otras apps aporta de verdad.

NO, SI...

- El token se añade «porque es un curso de blockchain».
- Una variable en un contrato haría lo mismo sin token.
- Solo sirve para especular, sin uso real dentro del proyecto.



PLANTILLA DE DECISIÓN (MEDIA PÁGINA, PARA EL AVANCE 1)

Respondan por escrito: (1) ¿Qué circula entre usuarios y por qué debe ser fungible? (2) ¿Una variable en un contrato lo resolvería sin token? (3) Si SÍ lleva token: suministro, emisión, quema y distribución. Si NO: por qué. Las dos respuestas valen igual; lo que no vale es un token decorativo. Es el mismo criterio de pertinencia de la Sesión 1.

¿MI PROYECTO LLEVA TOKEN? EN CINCO PREGUNTAS

#	PREGUNTA	SI LA RESPUESTA ES...
1	¿Qué circula entre usuarios, y por qué cada unidad debe ser idéntica a otra?	«Nada circula» → no lleva token.
2	¿Una variable o un mapping en un contrato haría lo mismo?	«Sí» → no lleva token: usen el mapping.
3	¿Aporta algo que billeteras y otras apps lo reconozcan (interoperabilidad)?	«No» → el estándar ERC-20 sobra.
4	Si lleva: ¿suministro, emisión, quema y distribución? (A.11)	Cada respuesta con su porqué, en una línea.
5	¿Quién puede cambiar las reglas después, y quién lo controla? (A.11c)	Si es una sola llave, justificar por qué.

EJEMPLO RESUELTO: UN REGISTRO DE DIPLOMAS

Un diploma no es intercambiable (pregunta 1: nada fungible circula) y un mapping lo resuelve (pregunta 2). Conclusión: no lleva ERC-20. Si acaso, un token no fungible, tema de la Sesión 11.



LA PREGUNTA DE LA SEMANA

¿Han aprobado alguna vez una cantidad ilimitada de un token a una aplicación? Revisen sus aprobaciones activas con una herramienta de revocación y cuenten qué encontraron.

PARA LA PRÓXIMA SESIÓN

- Entregar la evidencia del laboratorio 10 (token verificado + hashes).
- Completar la plantilla de decisión de tokenómica del proyecto (lleva token o no).
- Leer sobre NFTs e IPFS para la Sesión 11: la diferencia entre «el NFT» y «el archivo».
- Quien no haya empezado la nivelación de React: la Sesión 12 la asume por completo.



LO QUE SE LLEVAN DE HOY

Un token no vale por su código. Vale porque todo el ecosistema ya sabe hablarle.

Hoy construyeron algo fungible: cada unidad igual a otra. La próxima sesión, lo contrario: tokens únicos, y el problema de dónde vive de verdad la imagen.

SESIÓN 11 · NFTS, ERC-721/1155 Y ALMACENAMIENTO DESCENTRALIZADO